

KUBERNETES

How container orchestration actually works — the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. Why Orchestration?
2. Control Plane vs. Worker Nodes
3. A Pod: Containers That Share a Home
4. Deployment: Declare, Then Forget
5. A Service Is a Stable Front Door
6. Config Lives Outside the Image
7. Liveness vs. Readiness Probes
8. Horizontal Pod Autoscaler
9. Rolling Update: No Downtime
10. Cheat Sheet

Kubernetes builds directly on the container fundamentals from the Docker guide -- if any term here (image, layer, container) feels unfamiliar, that companion book is worth reading first.

CHAPTER 1

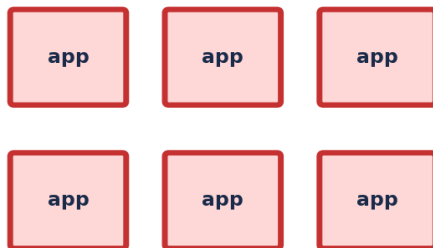
Why Orchestration?

Running one container is easy. Running hundreds, across dozens of machines, and keeping them all healthy, scaled correctly, and reachable -- by hand -- quickly becomes impossible.

Kubernetes flips the model: instead of imperatively deciding *which machine runs which container* yourself, you declare the **desired state** ("I want 6 healthy copies of this app, always") and Kubernetes continuously works to make reality match that declaration -- rescheduling, restarting, and rebalancing as needed, without you micromanaging any of it.

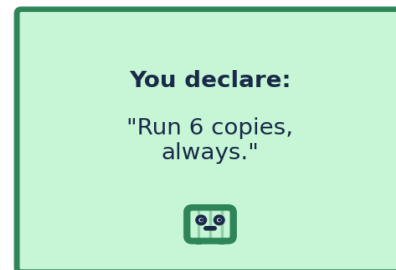
Why Orchestration?

Manually



Which machine? What if one crashes? How do you scale it?

With Kubernetes



Kubernetes figures out where, and keeps it that way.

IN SHORT: You declare desired state. Kubernetes' job is to continuously reconcile actual state toward it -- forever, automatically, without you watching.

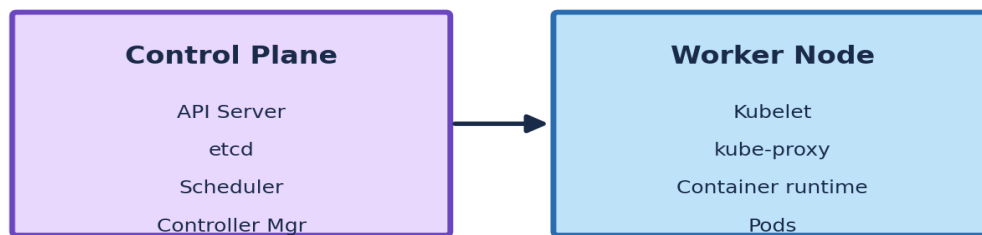
CHAPTER 2

Control Plane vs. Worker Nodes

Every Kubernetes cluster splits into two categories of machine. The **control plane** makes decisions: the API Server is the front door every other component talks through, etcd stores the entire cluster's actual state, the Scheduler decides which node a new Pod should run on, and the Controller Manager runs the reconciliation loops that keep actual state matching desired state.

Worker nodes are where your actual workloads run. Each runs a Kubelet (which talks to the control plane and starts/stops containers accordingly), kube-proxy (which handles the networking rules for Services), and the container runtime itself.

Control Plane vs. Worker Nodes



*The control plane decides WHAT should run and WHERE.
Worker nodes actually RUN it.*



IN SHORT: Control plane = the brain, making decisions. Worker nodes = the hands, actually running your containers.

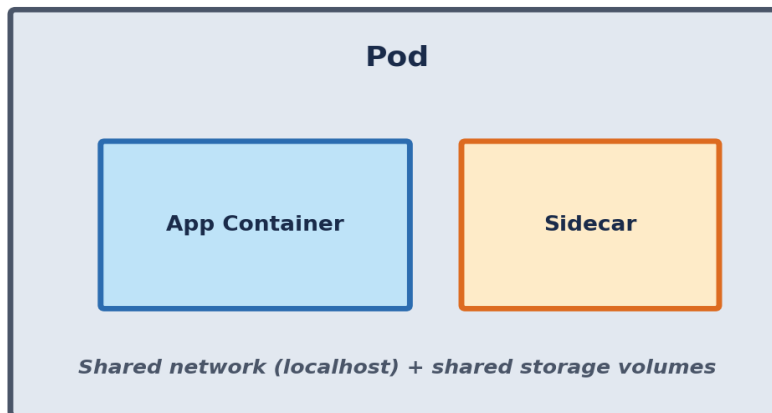
CHAPTER 3

A Pod: Containers That Share a Home

A Pod -- not a container -- is the smallest thing you actually deploy in Kubernetes. Most Pods hold just one container, but a Pod can hold several that are tightly coupled enough to need to run and scale together.

Containers within the same Pod share a network namespace (they can reach each other via localhost) and can share storage volumes. This is what makes the **sidecar pattern** possible -- a small helper container (a log shipper, a service mesh proxy) running alongside your main application container, sharing its network and disk.

A Pod: Containers That Share a Home



IN SHORT: One or more containers, always scheduled and scaled together, sharing a network identity. That's a Pod.

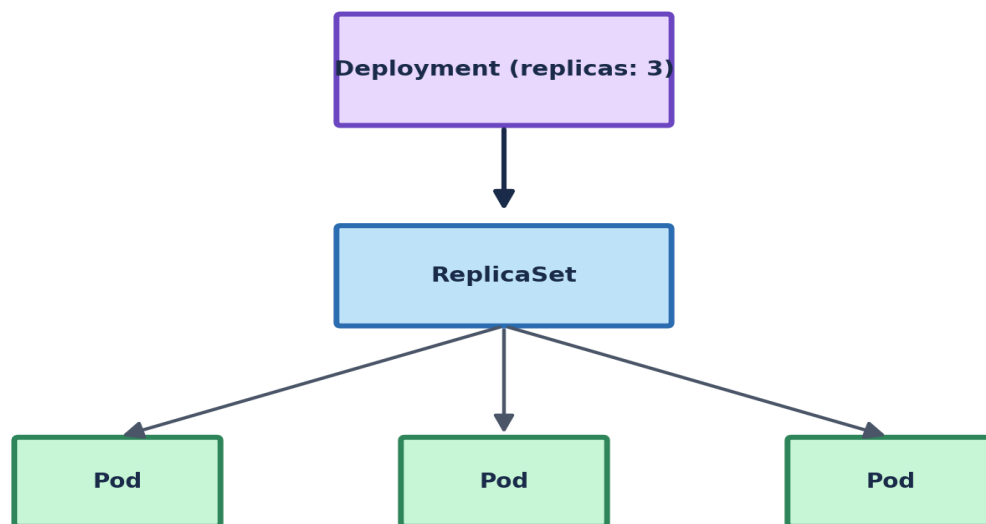
CHAPTER 4

Deployment: Declare, Then Forget

A Deployment is where you actually state your intent: which container image, how many replicas, how updates should roll out. You almost never create Pods directly.

Under the hood, a Deployment manages a ReplicaSet, and the ReplicaSet's only job is making sure the declared number of Pods actually exist. If a Pod crashes, gets evicted, or the node it's on dies, the ReplicaSet notices on its next reconciliation pass and creates a replacement -- no human has to be paged for that.

Deployment: Declare, Then Forget



If a Pod dies, the ReplicaSet notices and creates a replacement -- automatically.

IN SHORT: Deployment -> ReplicaSet -> Pods. Each layer's only job is keeping the layer below it matching what was declared.

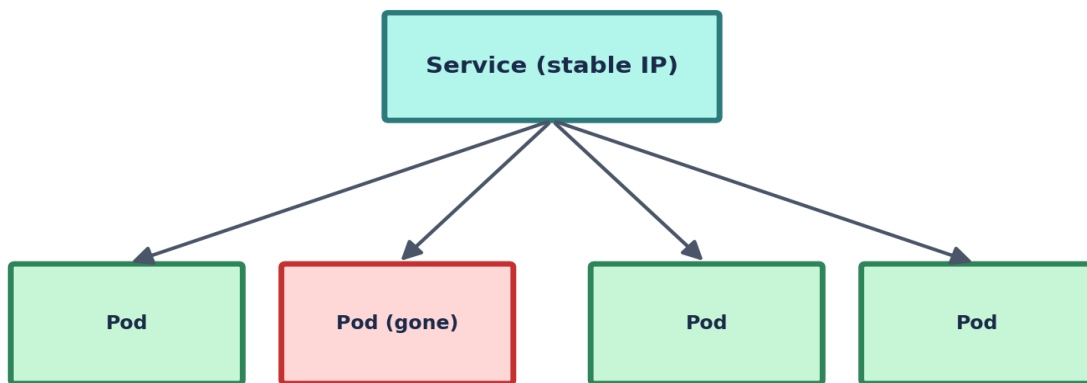
CHAPTER 5

A Service Is a Stable Front Door

Pod IP addresses are not something you should ever hold onto. A Pod can be rescheduled, restarted, or replaced at any moment, and it gets a brand-new IP every time.

A Service solves this by giving you one stable IP address (and DNS name) that automatically load-balances across whichever Pods currently match its label selector and are currently healthy. Client code always talks to the Service -- never directly to a specific Pod's IP.

A Service Is a Stable Front Door



Pod IPs come and go. The Service's IP never changes -- traffic is always routed to whichever Pods are currently healthy.

IN SHORT: Never hardcode a Pod's IP. Always talk to a Service -- it's the stable identity that survives Pods coming and going underneath it.

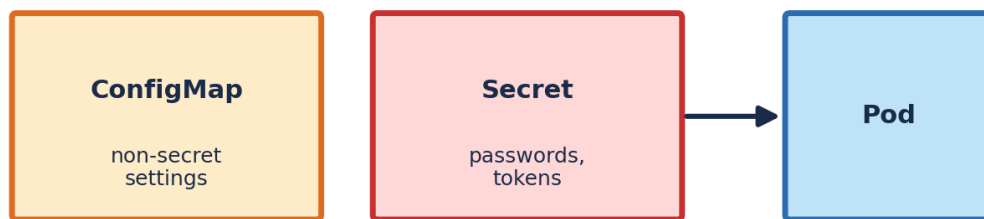
CHAPTER 6

Config Lives Outside the Image

Baking environment-specific configuration directly into a container image means rebuilding the image every time a setting changes across dev, staging, and production. Kubernetes gives you two objects specifically to avoid that.

A **ConfigMap** holds non-sensitive configuration -- feature flags, URLs, log levels. A **Secret** holds sensitive values -- passwords, API tokens, certificates (though, as covered in the Kubernetes Q&A file, a Secret's default protection is weaker than its name implies -- it's base64-encoded, not encrypted, unless you explicitly configure encryption at rest). Both can be mounted into a Pod as environment variables or files.

Config Lives Outside the Image



IN SHORT: Same image, different config per environment -- mounted in at runtime, not baked in at build time.

CHAPTER 7

Liveness vs. Readiness Probes

These two probes look similar but trigger completely different consequences when they fail, and mixing them up is one of the most common sources of self-inflicted production incidents in Kubernetes.

A failing **liveness** probe means "this container is broken beyond recovery" -- Kubernetes kills and restarts it. A failing **readiness** probe means "this container isn't ready for traffic *right now*" -- Kubernetes just stops routing Service traffic to it, without killing anything, and resumes sending traffic once it passes again.

Liveness vs. Readiness Probes

Liveness fails



*Container is killed
and restarted.*

Readiness fails



*Pod stays alive, but is
pulled out of Service
traffic until it recovers.*

IN SHORT: Liveness failure = restart the container. Readiness failure = just stop sending it traffic. Getting this backwards causes needless restarts of perfectly healthy Pods.

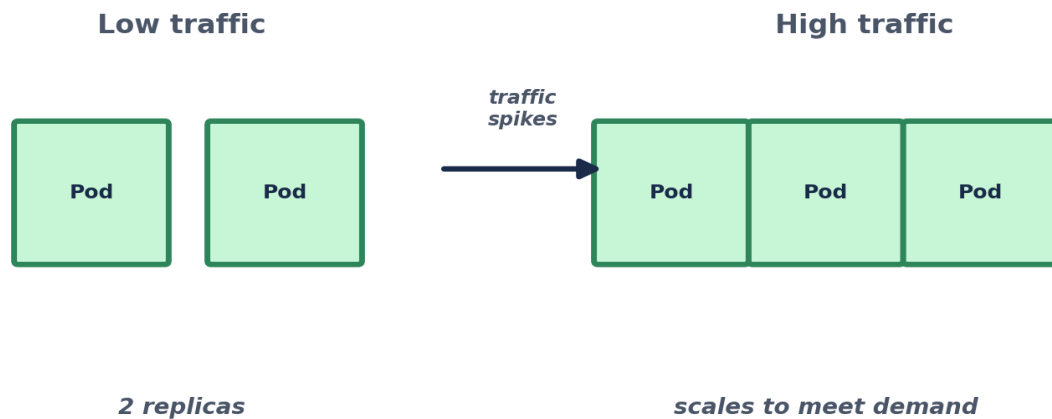
CHAPTER 8

Horizontal Pod Autoscaler

The Horizontal Pod Autoscaler watches a metric -- usually CPU or memory utilization, but it can watch custom application metrics too -- and automatically adjusts how many Pod replicas exist to keep that metric near a target value.

As traffic climbs and average CPU usage crosses the configured threshold, the HPA increases the replica count. As traffic falls back off, it scales back down -- all without a human deciding when to scale.

Horizontal Pod Autoscaler



IN SHORT: You set a target metric and a min/max replica range. The HPA handles the actual scaling decisions in between, continuously.

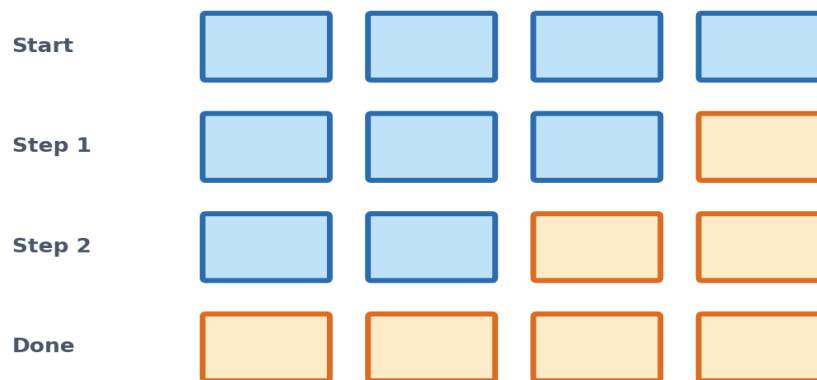
CHAPTER 9

Rolling Update: No Downtime

When you update a Deployment's image, Kubernetes doesn't kill every old Pod and start every new one at once -- that would mean a window with zero capacity serving traffic.

Instead, it performs a rolling update: replacing a small batch of old Pods with new ones, waiting for the new ones to pass their readiness probes, then moving on to the next batch -- controlled by `maxSurge` (how many extra Pods can exist temporarily) and `maxUnavailable` (how many can be down at once).

Rolling Update: No Downtime



Old version (blue) is replaced by new (orange) a few Pods at a time -- traffic never fully stops.

IN SHORT: Old and new versions briefly coexist during the rollout. Capacity never drops to zero, and a bad rollout can be rolled back the same gradual way.

CHAPTER 10

Cheat Sheet

The kubectl commands you'll reach for constantly.

KUBERNETES CHEAT SHEET



Pods & Deployments

```
kubectl get pods  
kubectl describe pod <name>  
kubectl apply -f file.yaml  
kubectl scale deploy/x --replicas=5
```

Debugging

```
kubectl logs -f <pod>  
kubectl exec -it <pod> -- bash  
kubectl get events  
kubectl top pods
```

Services & Networking

```
kubectl get svc  
kubectl port-forward svc/x 8080:80  
kubectl get endpoints
```

Rollouts

```
kubectl rollout status deploy/x  
kubectl rollout undo deploy/x  
kubectl rollout history deploy/x
```

Config

```
kubectl create configmap ...  
kubectl create secret generic ...  
kubectl get all -n <namespace>
```